RATE LIMITING

1. Why Rate Limiting Is Needed

Without rate limiting, anyone can send unlimited requests to your APIs.

Example attack:

Bot sends 10,000 requests / second

Possible results:

- Server CPU overload
- Database overload
- Redis overload
- API crash
- Login brute-force attacks

Example:

POST /login

Attacker tries 1000 passwords per second.

Rate limiting blocks this.

---

2. What Rate Limiting Does

It restricts how many requests a client can make within a time window.

Example rule:

100 requests / minute per IP

If user exceeds limit:

Server returns:

HTTP 429
Too Many Requests

---

3. Where Rate Limiting Is Applied

Usually on:

Public APIs

GET /products
GET /products/{id}

Authentication APIs

POST /login
POST /signup

Expensive APIs

search
checkout
order creation

---

4. Types of Rate Limiting

Fixed Window

Example:

100 requests per minute

Counter resets every minute.

Simple but not perfect.

Sliding Window

Requests counted in rolling time window.

More accurate.

Token Bucket (Most Common)

Requests consume tokens.

Example:

Bucket capacity = 10
Refill = 10 tokens / minute

Each request consumes a token.

If bucket empty → request rejected.

This is industry standard.

---

5. Best Library for Spring Boot

We will use:

Bucket4j

Why?

- Very popular
- High performance
- Works with Redis
- Easy integration
- Used in production systems

---

6. Rate Limiting Architecture

Request flow will become:

```
Client
  ↓
Rate Limiter Filter
  ↓
If allowed → Controller
If exceeded → 429 Error
```

So the request is blocked before reaching your services.

---

7. Example Rule for Advann

We can define something like:

```
/products        → 100 requests / minute
/product/{id}    → 100 requests / minute
/login           → 5 requests / minute
```

---

8. Implementation Steps

We will implement this in 3 small steps.

Step 1

Add dependency.

Step 2

Create RateLimitFilter.

Step 3

Apply limit per IP.

---

9. Step 1 – Add Dependency

Add this to pom.xml.

```
<dependency>
   <groupId>com.bucket4j</groupId>
   <artifactId>bucket4j-core</artifactId>
   <version>8.10.1</version>
</dependency>
```

Then run:

```
mvn clean install
```

10. Step 2 – Create Rate Limit Filter

Create class:

config/RateLimitFilter.java

Example implementation:

```
@Component
public class RateLimitFilter extends OncePerRequestFilter {

    private final Map<String, Bucket> bucketCache = new ConcurrentHashMap<>();

    private Bucket createNewBucket() {

        Bandwidth limit = Bandwidth.simple(100, Duration.ofMinutes(1));

        return Bucket.builder()
                .addLimit(limit)
                .build();
    }

    private Bucket resolveBucket(String ip) {
        return bucketCache.computeIfAbsent(ip, k -> createNewBucket());
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String ip = request.getRemoteAddr();

        Bucket bucket = resolveBucket(ip);

        if (bucket.tryConsume(1)) {
            filterChain.doFilter(request, response);
        } else {

            response.setStatus(429);
            response.getWriter().write("Too many requests. Please try later.");
        }
    }
}
```

11. What This Code Does

Each IP gets its own bucket.

Example:

192.168.1.10 → 100 requests/minute
192.168.1.20 → 100 requests/minute

Each request consumes 1 token.

If tokens finished:

HTTP 429 Too Many Requests

---

12. Testing Rate Limiting

Use Postman or curl.

Send many requests quickly.

Example:

GET /products

After limit reached:

429 Too Many Requests

---

13. Important Note for Production

This simple implementation stores buckets in memory.

In real production systems with multiple instances we store them in:

Redis

Later in Advann we can upgrade it to Redis distributed rate limiting.

Where You Should Add Rate Limiting

1. Add Dependency

Add the dependency in:

api-gateway/pom.xml

Example:

```
<dependency>
  <groupId>com.bucket4j</groupId>
  <artifactId>bucket4j-core</artifactId>
```

```
    <version>8.10.1</version>
</dependency>
```

Then rebuild the gateway service.

Where to Add the Rate Limiting Code

Create a configuration class inside the gateway:

```
api-gateway
  └── src/main/java
      └── com/advann/apigateway
          └── config
              └── RateLimitFilter.java
```

So create a new package:

config

inside the **api-gateway service**.

Final Folder Structure

Inside **api-gateway** it should look like this:

```
api-gateway
 └── src
     └── main
         └── java
             └── com
                 └── advann
                     └── apigateway
                         ├── config
                         │   └── RateLimitFilter.java
                         ├── controller
                         ├── gateway
                         └── ApiGatewayApplication.java
```

Why Gateway Is Correct Place

Example request flow:

```
User → GET /products
      ↓
API Gateway
      ↓
Rate Limit Check
      ↓
```

Allowed → product-service
Blocked → 429 Too Many Requests

So your microservices remain **clean and focused on business logic**.

Industry Architecture

Real systems do the same:

Client
↓
API Gateway
↓
Rate Limiter
↓
Microservices
↓
Database

Companies like:

- Netflix
- Amazon
- Uber

all enforce **rate limiting at the gateway layer**.

Next Step

So do this now:

1️⃣ Add dependency in

api-gateway/pom.xml

2️⃣ Create package

api-gateway/config

3️⃣ Add

RateLimitFilter.java

NOW WE WILL IMPLEMENT RATE LIMITTING IN ADVANN PROJECT

Step 1 — Add Dependency in API Gateway

Open:

api-gateway/pom.xml

Add Redis reactive dependency.

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

You probably already have:

spring-cloud-starter-gateway

If not, add it:

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

Then rebuild the gateway.

```
mvn clean install
```

Step 2 — Configure Redis in Gateway

Open:

api-gateway/src/main/resources/application.yml

Add Redis connection.

```
spring:

data:
  redis:
    host: redis
    port: 6379
```

If Redis runs locally:

```
spring:

data:
  redis:
    host: localhost
    port: 6379
```

Since you are using **Docker**, usually:

redis

is the container name.

```yaml
server:
  port: 8080

spring:
  application:
    name: api-gateway

  data:
    redis:
      host: redis
      port: 6379

  cloud:
    gateway:
      routes:

        - id: product-service
          uri: lb://product-service
          predicates:
            - Path=/product-service/**
          filters:
            - StripPrefix=1

        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/user-service/**
          filters:
            - StripPrefix=1

        - id: cart-service
          uri: lb://cart-service
          predicates:
            - Path=/cart-service/**
          filters:
            - StripPrefix=1

        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/order-service/**
          filters:
            - StripPrefix=1

        - id: payment-service
          uri: lb://payment-service
          predicates:
```

```
        - Path=/payment-service/**
      filters:
        - StripPrefix=1

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://eureka-server:8761/eureka/

jwt:
  secret: 12345678901234567890123456789012
```

Step 3 — Create Rate Limiter Configuration

Create a new package inside gateway:

```
api-gateway
 └── src/main/java
     └── com/advann/apigateway/config
```

Create class:

RateLimiterConfig.java

Code:

```
package com.advann.apigateway.config;

import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import reactor.core.publisher.Mono;
import org.springframework.web.server.ServerWebExchange;

@Configuration
public class RateLimiterConfig {

    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> Mono.just(getClientIP(exchange));
    }

    private String getClientIP(ServerWebExchange exchange) {

        String ip = exchange.getRequest()
```

```
            .getHeaders()
            .getFirst("X-Forwarded-For");

        if (ip == null) {
            ip = exchange.getRequest()
                    .getRemoteAddress()
                    .getAddress()
                    .getHostAddress();
        }

        return ip;
    }
}
```

What this does

Rate limit will be applied **per client IP**.

Example:

192.168.1.10 → limit applied
192.168.1.11 → separate limit

Step 4 — Configure Rate Limiting in Gateway Routes

Open:

api-gateway/src/main/resources/application.yml

Add the **RequestRateLimiter filter**.

```
server:
  port: 8080

spring:
  application:
    name: api-gateway

  data:
    redis:
      host: redis
      port: 6379
```

```yaml
cloud:
  gateway:
    routes:

      - id: product-service
        uri: lb://product-service
        predicates:
          - Path=/product-service/**
        filters:
          - StripPrefix=1
          - name: RequestRateLimiter
            args:
              key-resolver: "#{@userKeyResolver}"
              redis-rate-limiter.replenishRate: 10
              redis-rate-limiter.burstCapacity: 20

      - id: user-service
        uri: lb://user-service
        predicates:
          - Path=/user-service/**
        filters:
          - StripPrefix=1
          - name: RequestRateLimiter
            args:
              key-resolver: "#{@userKeyResolver}"
              redis-rate-limiter.replenishRate: 5
              redis-rate-limiter.burstCapacity: 10

      - id: cart-service
        uri: lb://cart-service
        predicates:
```

```
          - Path=/cart-service/**
        filters:
          - StripPrefix=1

      - id: order-service
        uri: lb://order-service
        predicates:
          - Path=/order-service/**
        filters:
          - StripPrefix=1

      - id: payment-service
        uri: lb://payment-service
        predicates:
          - Path=/payment-service/**
        filters:
          - StripPrefix=1

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://eureka-server:8761/eureka/

jwt:
  secret: 12345678901234567890123456789012
```

What this configuration does

For **product-service**

10 requests per second
burst up to 20

For **user-service**

5 requests per second
burst up to 10

Other services currently **have no rate limiting**.

Request Flow After This

Client
↓
API Gateway
↓
Redis RateLimiter
↓
Allowed → product-service
Blocked → 429 Too Many Requests

After Adding This

Restart your containers:

```
mvn clean install -DskipTests

docker compose down
docker compose up --build -d
```

Quick Test

Send many requests quickly to:

http://localhost:8080/product-service/products

After the limit is exceeded, you should receive:

HTTP 429
Too Many Requests

How to check or debug steps?

6. Restarting the System

After configuration changes, containers must be rebuilt.

Commands used:

```
mvn clean install -DskipTests
docker compose down
docker compose up --build -d
```

---

7. Testing the Rate Limiter

API tested:

http://localhost:8080/product-service/api/products

Basic request:

```
curl -i http://localhost:8080/product-service/api/products
```

Example response:

```
HTTP/1.1 200 OK
X-RateLimit-Remaining: 1
X-RateLimit-Burst-Capacity: 2
X-RateLimit-Replenish-Rate: 1
```

Headers confirm that the **Redis rate limiter is active**.

---

8. Sending Multiple Requests

To test rate limiting, multiple requests were sent:

```
for /L %i in (1,1,10) do curl -i http://localhost:8080/product-service/api/products
```

Expected responses:

```
HTTP 200
HTTP 200
HTTP 429 Too Many Requests
HTTP 429 Too Many Requests
```

Reason:

First two requests consume tokens from the bucket.

Remaining requests are rejected.

---

9. Gateway Logs Verification

Gateway debug logging was enabled.

Configuration:

```
logging:
  level:
    org.springframework.cloud.gateway: DEBUG
```

Log output confirmed rate limiting:

RedisRateLimiter : response: Response{allowed=false}

Meaning:

allowed=false

The request exceeded the rate limit and was blocked.

---

10. Response Headers Added by Gateway

Example headers returned by Spring Cloud Gateway:

X-RateLimit-Remaining
X-RateLimit-Burst-Capacity
X-RateLimit-Replenish-Rate

Example:

X-RateLimit-Remaining: 1
X-RateLimit-Burst-Capacity: 2
X-RateLimit-Replenish-Rate: 1

These headers help clients understand their remaining request quota.

---

11. Final Architecture After Implementation

Current Advann backend request flow:

Client
↓
API Gateway
↓
Redis Rate Limiter
↓
JWT Authentication Filter
↓
Microservice
↓
Database

Components used:

- Spring Cloud Gateway
- Redis
- Docker
- Eureka
- JWT authentication

---

12. Outcome

The system now prevents excessive API usage and protects backend services.

Rate limiting is implemented using:

Spring Cloud Gateway + RedisRateLimiter